

Assignment 3 COMP 551

Leen Assaf, Karl Wehbe, and Sebastian Moseres

1 Abstract

In this project, we implement a multilayer perceptron (MLP) from scratch to classify the Kuzushiji-MNIST dataset, which contains 70,000 28×28 images of handwritten Japanese characters across 10 classes. We explore various design choices—such as network depth, activation functions, and regularization techniques—to understand their impact on classification accuracy. Our extensive hyperparameter tuning revealed that a two-layer MLP with hidden layers of **256** and **128 units**, a learning rate of **0.05**, and a batch size of **32** produced the best performance, achieving a mean test score of approximately **94.1%**. We found that adding hidden layers and using effective activation functions (e.g., **LeakyReLU** and **Sigmoid**) can significantly boost accuracy compared to simpler models, and that a small amount of L2 regularization ($\lambda = 0.001$) is key to optimizing generalization. We also implemented a convolutional neural network (CNN) using TensorFlow and Keras and compared its performance against our custom MLP models. The CNN clearly outperformed all MLP variants by achieving perfect validation accuracy, demonstrating its superior ability to extract spatial features from image data. Overall, this work provides valuable insights into the effects of fundamental design choices and highlights the strengths and limitations of MLPs for image classification tasks.

2 Introduction

Deep learning has revolutionized the field of computer vision, enabling significant advancements in image classification tasks. Among the various deep learning models, multilayer perceptrons (MLPs) serve as fundamental building blocks that demonstrate the power of representation learning. In this project, we implement MLPs from scratch and apply them to the classification of Kuzushiji-MNIST (KMIST) images, a dataset containing handwritten Japanese characters from the Edo period (Clanuwat et al., 2018).

The KMIST dataset poses unique challenges compared to the standard MNIST dataset of handwritten digits. The characters in KMIST are more complex, with intricate structures and variations in writing styles, making it a more challenging benchmark for evaluating neural network performance. By working with this dataset, we gain insights into how different neural network architectures and configurations handle complex pattern recognition tasks.

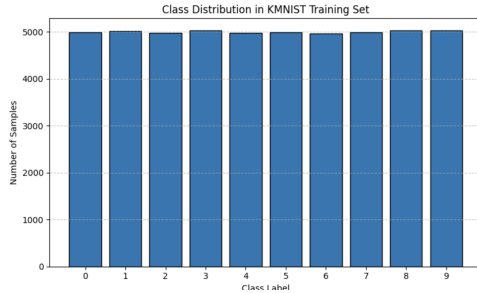
Our implementation focuses on exploring several key aspects of neural network design: the impact of network depth, the choice of activation functions, and the effect of regularization techniques. We systematically evaluate how these factors influence the model's ability to learn meaningful representations from image data and make accurate predictions. Additionally, we compare our custom MLP implementations with a convolutional neural network (CNN) built using modern deep learning frameworks, highlighting the advantages of specialized architectures for image processing tasks.

3 Datasets

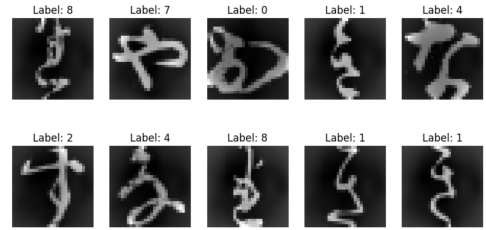
For this project, we worked with the Kuzushiji-MNIST (KMIST) dataset. A modern alternative to the original MNIST dataset which is a large database of handwritten digits that is used for training various machine learning models on image classification tasks. KMIST

contains 70,000 images, each showing a handwritten Japanese character. These characters come from a set of 10 different Hiragana classes, with 60,000 datapoints allocated for training and 10,000 for testing.

To get a better understanding of the data, we started with some basic exploratory data analysis. We first plotted a histogram of label counts and visualized a sample of random images from the training set. We realized that the characters are much more complex than the digits in MNIST, with high variation in writing styles. Some classes appear visually similar to others, which might increase the difficulty for the model to distinguish between them.



((a)) Label distribution in the KMNIST training set



((b)) Sample images from the training set

Figure 1: Exploratory analysis of the KMNIST dataset

We also preprocessed the images by flattening them into 784-dimensional vectors and standardized them, so that each feature has zero mean and 1 unit variance. This was important to stabilize and speed up training, especially in neural networks when poor scaling can cause gradients to explode.

Overall, this dataset provided a challenge for our different models, as it made the image classification task harder with the complex and unfamiliar character shapes.

4 Results

4.1 Hyperparameter Tuning for Single-Layer and Two-Layer MLP Models

To get the best performance out of our MLP models, we experimented with a few key hyperparameters: the network architecture (specifically, the number of hidden units in each layer), the batch size, and the learning rate. Our main goal was to maximize the test score while ensuring the model captured the underlying data patterns effectively.

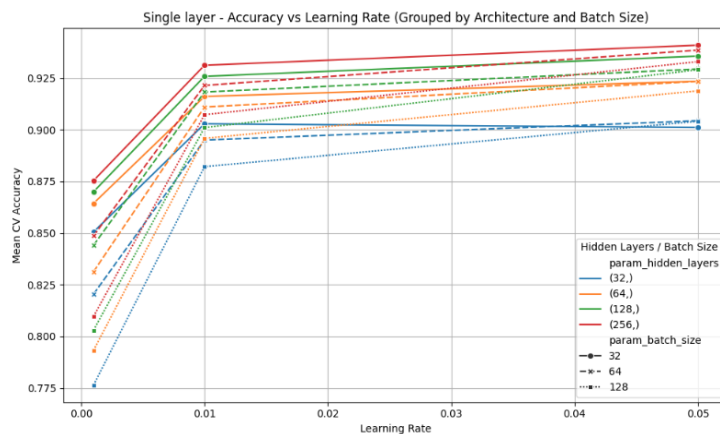


Figure 2: Single-Layer MLP Hyperparameter Tuning

Table 1: Top 5 Hyperparameter Combinations

Mean Test Score	Hidden Layer	Learning Rate	Batch Size
0.94100	256	0.05	32
0.93658	256	0.05	64
0.93410	256	0.05	128
0.93384	128	0.05	32
0.93204	256	0.01	32

For the single-layer MLP, as shown in Figure 2 and Table 1, we found that using a hidden layer with **256 units**, a learning rate of **0.05**, and a batch size of **32** produced the best results, with a mean test score of **0.94100 (or 94.1%)**. Increasing the batch size or lowering the learning rate led to slightly lower performance, and using fewer hidden units (128 units) didn't match the performance of the 256-unit configuration.

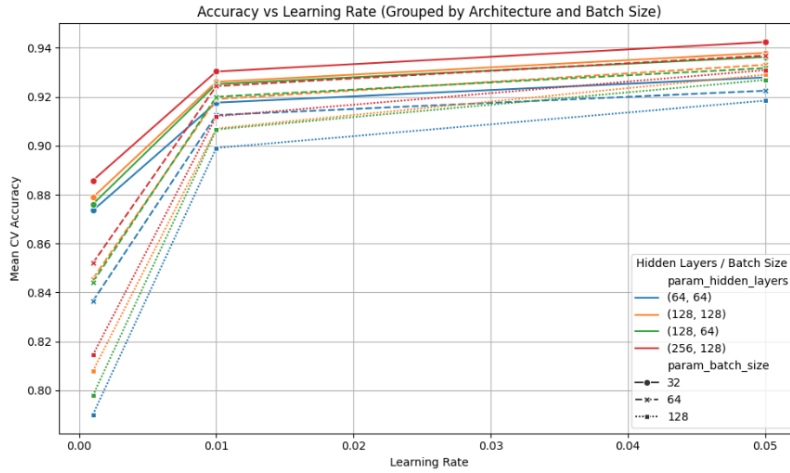


Figure 3: Two-Layer MLP Hyperparameter Tuning

Table 2: Top 5 Hyperparameter Combinations for the 2-Layer MLP

Mean Test Score	Hidden Layers	Learning Rate	Batch Size
0.94240	(256, 128)	0.05	32
0.93798	(128, 128)	0.05	32
0.93686	(256, 128)	0.05	64
0.93630	(128, 64)	0.05	32
0.93310	(128, 128)	0.05	64

Similarly, for the two-layer MLP, Figure 3 and Table 2 reveal that the optimal setup is achieved with two hidden layers where the first layer has **256 units** and the second has **128 units**. This configuration, also using a learning rate of **0.05** and a batch size of **32**, reached a mean test score of **0.94240**. Other configurations with different hidden layer sizes performed slightly worse, confirming that a deeper network with a larger first layer followed by a slightly smaller second layer is particularly effective for this task..

Overall, the accuracy differences between the various hyperparameter settings are minimal, indicating that both model configurations are robust. Based on our testing, the recommended hyperparameters are:

- **Single-layer MLP:** a hidden layer with **256 units**, a batch size of **32**, and a learning rate of **0.05**.
- **2-Layer MLP:** two hidden layers with **256** and **128 units** respectively, a batch size of **32**, and a learning rate of **0.05**.

This combination not only maximizes test performance but also provides a stable foundation that will be used for further experiments.

4.2 Model Architecture Comparison

In this section, we take a closer look at the performance of three different multilayer perceptron (MLP) models. First, we consider an MLP with **no hidden layers**, which serves as our **baseline**. Then, we examine an MLP with a **single hidden layer**, which can capture more complex patterns than the baseline model. Finally, we explore an MLP with **two hidden layers**, adding even more capacity to learn subtle relationships in the data. We compare these models based on both **validation** and **test accuracies** to see how increasing network depth affects performance.

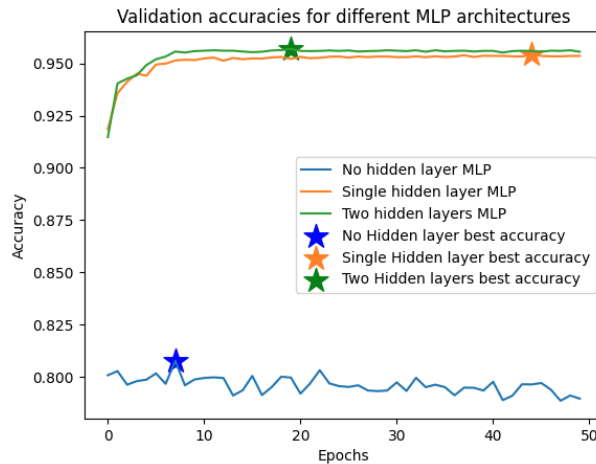


Figure 4: Validation Accuracies for MLPs with Different Activation Functions

Table 3: Validation and Test Accuracies for MLP Architectures

Model	Validation Accuracy	Test Accuracy
No hidden layers	80.75%	66.96%
Single hidden layer	95.44%	89.31%
Two hidden layers	95.69%	90.05%

As shown in Figure 6 and Table 3, there's a clear **performance gap** between the MLP with **no hidden layers** and those with hidden layers. This indicates that adding hidden layers significantly boosts the model's ability to capture **complex patterns** in the data. The jump from **no hidden layers** to a **single hidden layer** is especially notable. However, the improvement from one

hidden layer to two hidden layers is relatively small, suggesting that while extra depth can help capture additional subtleties, it doesn't always translate to a **substantial increase in accuracy**, at least not with this dataset.

4.3 *K-fold Cross Validation*

To further evaluate the **robustness** of our best-performing two-layer MLP model, we performed **5-fold cross-validation** on the training set. This approach allowed us to test the model's **consistency** across different splits of the data and ensure that its performance wasn't dependent on a particular train/validation split.

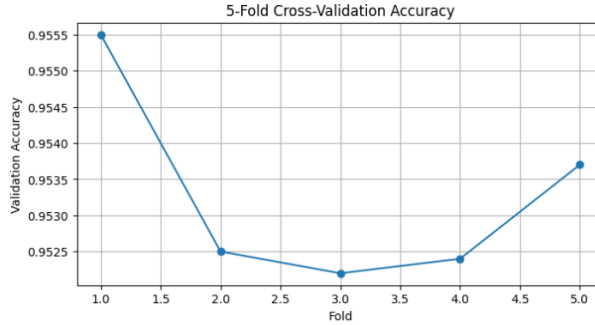


Figure 5: 5-fold cross validation accuracies

The model was trained with **256 and 128 hidden units**, a learning rate of **0.05**, and a batch size of **32**—the same hyperparameters identified as optimal in previous tuning. The validation accuracies across the five folds were **95.55%**, **95.25%**, **95.22%**, **95.24%**, and **95.37%**, resulting in an average cross-validation accuracy of approximately **95.33%**. These results suggest that our model **generalizes well** and performs consistently across different subsets of the data. The small variation in accuracy between folds also indicates that the training process is **stable** and not overly sensitive to data partitioning.

4.4 *Effects of Different Activation Functions*

In this section, we compare the performance of our 2-layer MLP models using different activation functions. We tested four setups: one with **ReLU** in the hidden layers and **Softmax** in the output, another with **LeakyReLU** and **Softmax**, a third with **ReLU** and **Sigmoid**, and finally one that combines **LeakyReLU** and **Sigmoid**. Figure 6 shows the validation accuracy curves for these models, while Table 4 summarizes the key results.

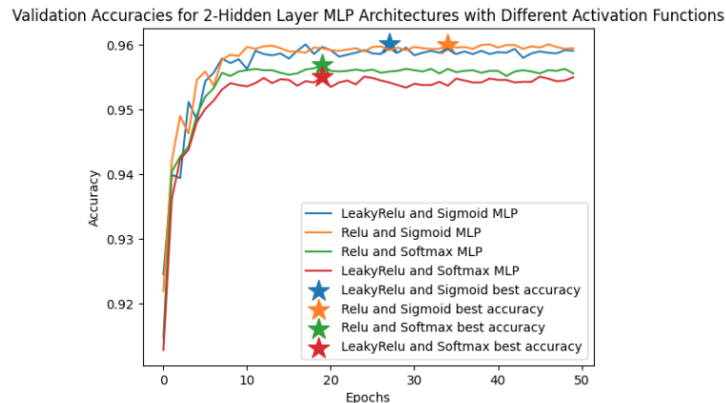


Figure 6: Validation Accuracies for 2-Hidden layer MLPs with Different Activation Functions

Table 4: Validation & Test Accuracies for MLPs using Different Activation Functions

Model	Best Validation Accuracy	Test Accuracy
ReLU and Softmax	95.69%	90.05%
LeakyReLU and Sigmoid	96.02%	90.18%
Leaky and Softmax	95.51%	90.00%
ReLU and Sigmoid	96.01%	90.03%

Looking at the numbers, the model using **LeakyReLU and Sigmoid** comes out on top, with a best validation accuracy of **96.02%** and a test accuracy of **90.18%**. This suggests that **LeakyReLU** might help by providing a more reliable gradient flow in the hidden layers—thus avoiding issues like the dying ReLU problem—while **Sigmoid** in the output seems to model the classification probabilities effectively. Although the other configurations (ReLU and Softmax, as well as ReLU and Sigmoid) performed well too, their scores are just slightly lower. These small differences hint that while activation functions do impact performance, the improvements are subtle and might be more suited to this particular dataset rather than showing a broad, definitive trend.

Overall, even though the differences are minor, the combination of **LeakyReLU and Sigmoid** appears to be the most effective choice for our experiment.

4.5 Effect of L2 Regularization on a 2-Layer MLP

We extended our **2-hidden layer MLP** model (using ReLU and Softmax) by adding **L2 regularization** to the network weights in all layers (hidden and output). We applied the penalty term $\lambda \|W^{(\ell)}\|_2^2$ and treated λ as a **tunable hyperparameter**. The table below summarizes our results:

Table 5: Effect of L2 Regularization on a 2-Layer MLP

λ	Training Accuracy	Validation Accuracy	Test Accuracy
0.1	80.42%	79.97%	42.95%
0.05	86.98%	86.15%	66.27%
0.01	96.30%	94.05%	85.44%
0.005	98.59%	95.37%	88.99%
0.001	99.98%	96.06%	90.25%

As shown in the table, as the regularization strength increases (i.e., higher λ values), the model’s performance—especially on the validation and test sets—drops dramatically. In contrast, using a very small amount of L2 regularization ($\lambda = 0.001$) results in the best performance, with a training accuracy of **99.98%**, validation accuracy of **96.06%**, and test accuracy of **90.25%**. These findings highlight that a minimal regularization works best in this case.

4.6 Experimenting with Different Fully Connected Layer Sizes in the CNN

To find the best setup for our CNN, we experimented with different numbers of units in the two fully connected layers. We tested models with hidden units chosen from $\{32, 64, 128, 256\}$ and selected the best configuration based on validation accuracy. Each model was trained using an early stopping callback (with a minimum improvement threshold of **0.0001**), a learning rate of **0.01**, and a batch size of **32**. The results are shown in Table 6.

Table 6: Validation and Test Accuracies for Different Fully Connected Layer Sizes in the CNN

Hidden Units	Validation Accuracy	Test Accuracy
32	99.04%	92.66%
64	99.08%	92.68%
128	99.56%	93.71%
256	99.76%	94.78%

Looking at the table, the model with **256 hidden units** achieved the highest validation accuracy (**99.76%**) and the best test accuracy (**94.78%**). Although the differences compared to the other configurations are small, the **256-unit** model benefits from increased capacity, helping it learn more complex patterns and generalize better.

4.7 Comparison of MLP and CNN Performance

In this section, we compare the performance of various MLP models against the same **Convolutional Neural Network (CNN)** with 256 hidden units (best configuration) from the previous experiment. The models are evaluated based on their best **validation** and **test accuracies**, clearly showing the gap between fully connected architectures and those that leverage convolutional layers.

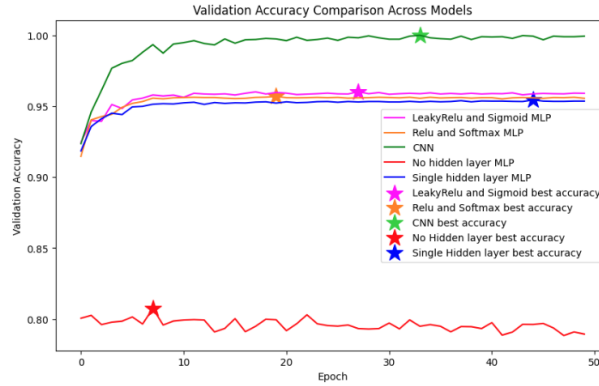


Figure 7: Validation and Test Accuracy Curves for MLP and CNN Models

Table 7: Validation and Test Accuracies for MLP and CNN Models

Model	Best Validation Accuracy	Test Accuracy
MLP with no hidden layer	80.75%	66.96%
MLP with single hidden layer	95.44%	89.31%
ReLU & Softmax MLP	95.69%	90.05%
LeakyReLU & Sigmoid MLP	96.02%	90.18%
CNN	100.00%	95.71%

As shown in Figure 7 and summarized in Table 7, the **CNN** model clearly **outperforms** all the MLP variants. While the MLP models achieve validation accuracies ranging from **80.75%** (for the simplest architecture with no hidden layers) up to about **96%** for more complex configurations, the CNN reaches a perfect validation accuracy of **100.00%** and a test accuracy of

95.71%. This substantial performance boost demonstrates the advantage of using **convolutional layers** to capture spatial features in image data—capabilities that fully connected MLPs simply cannot match.

Overall, the results strongly suggest that for tasks involving image data, a **CNN offers a significant edge** over MLP architectures, leading to much better generalization and robustness on unseen data.

5 Discussion and Conclusion

This project gave us a much deeper understanding of how multilayer perceptrons work and how different design choices affect their performance. One of the most important findings was how sensitive the MLP's accuracy is to hyperparameters, including network architecture, learning rate, batch size and regularization strength. Even small changes in these values could lead to noticeable differences in both training and test performance.

Using grid search, we explored a range of hyper parameter to identify the most effective configuration, we found that a two-layer MLP with 256 and 128 hidden units, a learning rate of 0.05, and a batch size of 32 increased the accuracy by approximately 10%.

To further confirm the reliability of our results, we conducted 5-fold cross-validation using the best-performing MLP configuration. The model achieved an average validation accuracy of approximately 95.34%, with very little variation across folds. This consistency suggests that our chosen hyperparameters generalize well across different subsets of the training data, and that the model's strong performance is not simply due to a favorable data split. These results reinforce the stability of the MLP and validate the effectiveness of our architecture and training choices.

Our experiments demonstrated that increasing the network depth and capacity lead to better results at first, but the impact decreases as the model becomes deeper. While adding a second hidden layer slightly enhanced validation and test accuracy, the gain over a well-tuned single-layer MLP was fairly small, suggesting that there may be diminishing returns for very deep MLPs on this particular task. On the other hand, experimenting with different activation functions showed that certain combinations, like ReLU in the hidden layers and Sigmoid in the output layer, gave slightly better results, although the differences weren't huge.

Regularization also played an important role in avoiding overfitting. Without it, the model would often perform very well on the training set but worse on validation and test sets. We found that a small amount of L2 regularization ($\lambda = 0.001$) improved generalization and gave us the best test accuracy for the MLP.

Finally, comparing our custom-built MLPs to a CNN implemented in Keras really highlighted how much more powerful convolutional layers are for image classification. The CNN outperformed all MLPs by a clear margin, despite having fewer trainable parameters in the fully connected layers. This makes sense since CNNs are designed to extract spatial features, which is especially useful in image classification.

Overall, this project gave us valuable hands-on experience with building, training, and evaluating neural networks, as well as a better appreciation of their strengths and limitations.

6 Statement of contribution

Leen Assaf: Task 1, Task 2, Task 3, Report

Karl Wehbe: Task 1, Task 2, Task 3, Report

Sebastian Moseres: Task 1, Task 2, Task 3, Report